

Okay, let's dive into the C code solutions and explanations for those sliding window problems.

1. Finding the Maximum Sum of a Subarray of a Given Size (k)

Problem Explanation:

Given an array of integers (arr) and a positive integer k (subarray size), the task is to find the subarray of size k that has the maximum sum.

Example:

Array: [1, 4, 2, 10, 2, 3, 1, 0, 20]

k = 4

Output: Maximum sum subarray is [2, 10, 2, 3] with sum 17.

[content_copy](#)

[download](#)

Use code with caution.

Sliding Window Solution Explanation:

We can use a sliding window of size k to efficiently solve this. The idea is:

Calculate the sum of the first k elements: This forms our initial window and its sum.

Slide the window one element at a time:

To slide, we subtract the element that is now leaving the window (the leftmost element of the previous window).

And add the element that is now entering the window (the element immediately to the right of the previous window).

Keep track of the maximum sum: As we slide the window and calculate the current window's sum, we compare it with the maximum sum found so far and update the maximum if the current sum is greater.

This avoids recalculating the sum of the entire window in each step, making it much more efficient than a brute-force approach.

C Code Solution:

```
#include <stdio.h>
```

```
#include <limits.h> // For INT_MIN
```

```

int maxSubarraySum(int arr[], int n, int k) {
    if (k > n) {
        printf("Invalid Input: k is greater than array size\n");
        return INT_MIN; // Or handle error as needed
    }

    int maxSum = INT_MIN; // Initialize maxSum to the smallest possible integer
    int currentSum = 0;

    // Calculate sum of the first window of size k
    for (int i = 0; i < k; i++) {
        currentSum += arr[i];
    }
    maxSum = currentSum; // Initial maximum is the sum of the first window

    // Slide the window through the rest of the array
    for (int i = k; i < n; i++) {
        currentSum -= arr[i - k]; // Subtract the element leaving the window
        currentSum += arr[i];     // Add the element entering the window
        if (currentSum > maxSum) {
            maxSum = currentSum; // Update maxSum if currentSum is greater
        }
    }

    return maxSum;
}

```

```

int main() {
    int arr[] = {1, 4, 2, 10, 2, 3, 1, 0, 20};
    int n = sizeof(arr) / sizeof(arr[0]);
    int k = 4;

    int maxSum = maxSubarraySum(arr, n, k);
    if (maxSum != INT_MIN) {
        printf("Maximum sum of subarray of size %d is: %d\n", k, maxSum);
    }
}

```

```

    return 0;
}

```

content_copy

download

Use code with caution.

C

Explanation of the C Code:

`maxSubarraySum(int arr[], int n, int k)` function:

Takes the integer array `arr`, its size `n`, and the subarray size `k` as input.

Error Handling: Checks if `k` is valid (not greater than array size). If invalid, prints an error message and returns `INT_MIN` (you can choose to handle errors differently, like returning an error code or exiting).

Initialization:

`maxSum = INT_MIN`:: Initializes `maxSum` to the smallest possible integer to ensure that any valid subarray sum will be greater.

`currentSum = 0`:: Initializes `currentSum` to 0.

First Window Sum: The first for loop (`for (int i = 0; i < k; i++)`) calculates the sum of the first `k` elements and stores it in `currentSum`. This is the sum of the initial window. `maxSum` is initially set to this `currentSum`.

Sliding Window Loop: The second for loop (`for (int i = k; i < n; i++)`) iterates from the `k`-th element to the end of the array. In each iteration:

`currentSum -= arr[i - k]`:: Subtracts the element that is now outside the left end of the window (element at index `i - k`).

`currentSum += arr[i]`:: Adds the element that is now entering the right end of the window (element at index `i`).

`if (currentSum > maxSum)`: Compares the `currentSum` with the `maxSum` found so far. If `currentSum` is greater, it updates `maxSum`.

Return Value: Returns the `maxSum` found.

`main()` function:

Sets up an example array `arr` and subarray size `k`.

Calls `maxSubarraySum()` to get the maximum sum.

Prints the result if a valid sum was returned (not `INT_MIN`).

2. Finding the Longest Substring with Unique Characters

Problem Explanation:

Given a string (str), find the length of the longest substring that contains only unique characters (no character is repeated within the substring).

Example:

String: "abcabcbb"

Output: Longest substring with unique characters is "abc", length is 3.

String: "bbbbbb"

Output: Longest substring with unique characters is "b", length is 1.

String: "pwwkew"

Output: Longest substring with unique characters is "wke", length is 3.

Note that "pwke" is a subsequence, not a substring.

content_copy

download

Use code with caution.

Sliding Window Solution Explanation:

We can use a sliding window approach combined with a data structure to track characters within the current window.

Initialize a window: Start with an empty window (left and right pointers at the beginning of the string).

Expand the window (move right pointer):

Move the right pointer to the right, one character at a time.

For each character, check if it's already present in the current window.

We can use a frequency array (or a hash map) to efficiently track characters in the window.

If the character is unique (not in the window):

Add it to the window (update frequency array).

Update the maximum length found so far if the current window's length is greater.

If the character is NOT unique (already in the window):

We have a duplicate, so we need to shrink the window from the left until the duplicate character is removed from the window.

Move the left pointer to the right, removing characters from the window (decrementing their counts in the frequency array) until the duplicate character is no longer in the window.

Then, you can add the current (duplicate) character to the window and continue expanding the window from the right.

C Code Solution:

```
#include <stdio.h>
#include <string.h>
#include <limits.h>

int longestUniqueSubstring(char str[]) {
    int n = strlen(str);
    int maxLength = 0;
    int left = 0;
    int right = 0;
    int charIndex[256]; // Array to store last index of each character (ASCII)

    for (int i = 0; i < 256; i++) {
        charIndex[i] = -1; // Initialize all indices to -1 (character not seen yet)
    }

    while (right < n) {
        char currentChar = str[right];

        // If the current character is already in the window (charIndex[currentChar] >= left)
        if (charIndex[currentChar] >= left) {
            // Shrink the window from the left
            left = charIndex[currentChar] + 1; // Move left pointer to just after the previous
            occurrence
        }

        charIndex[currentChar] = right; // Update the last seen index of the current character
        maxLength = (right - left + 1 > maxLength) ? (right - left + 1) : maxLength; // Update
        maxLength
        right++; // Expand the window to the right
    }

    return maxLength;
}
```

```

}

int main() {
    char str1[] = "abcabcbb";
    char str2[] = "bbbbbb";
    char str3[] = "pwwkew";

    printf("Longest unique substring length for \"%s\": %d\n", str1, longestUniqueSubstring(str1));
// Output: 3
    printf("Longest unique substring length for \"%s\": %d\n", str2, longestUniqueSubstring(str2));
// Output: 1
    printf("Longest unique substring length for \"%s\": %d\n", str3, longestUniqueSubstring(str3));
// Output: 3

    return 0;
}

```

content_copy
download
Use code with caution.
C

Explanation of the C Code:

`longestUniqueSubstring(char str[])` function:

Takes the string `str` as input.

Initialization:

`maxLength = 0`; Initializes `maxLength` to 0.

`left = 0`;, `right = 0`;: Initialize left and right pointers of the sliding window to the beginning of the string (index 0).

`int charIndex[256]`;: Creates an integer array `charIndex` of size 256 (assuming ASCII characters). This array will store the last seen index of each character within the current window. Initialized to -1 for all characters.

Sliding Window Loop: The while (`right < n`) loop iterates through the string using the right pointer.

`char currentChar = str[right]`;: Gets the current character at the right pointer.

Duplicate Check: if (`charIndex[currentChar] >= left`): Checks if the `currentChar` has already been seen in the current window. `charIndex[currentChar]` stores the last seen index of `currentChar`. If this index is greater than or equal to `left`, it means the character is within the current window (from `left` to `right - 1`).

Shrink Window: If a duplicate is found, `left = charIndex[currentChar] + 1`; moves the left pointer to just after the previous occurrence of the duplicate character. This effectively shrinks the window from the left, removing the previous occurrence of the duplicate and all characters to its left from the window.

`charIndex[currentChar] = right`:: Updates the `charIndex` array with the current right index as the last seen index for `currentChar`.

`maxLength = (right - left + 1 > maxLength) ? (right - left + 1) : maxLength`:: Calculates the current window length (`right - left + 1`) and updates `maxLength` if the current window is longer.

`right++`:: Expands the window by moving the right pointer to the next character.

Return Value: Returns the `maxLength`.

`main()` function:

Sets up example strings `str1`, `str2`, `str3`.

Calls `longestUniqueSubstring()` for each string and prints the result.

3. Finding Subarrays that Meet a Condition (Example: Subarrays with Sum K)

Problem Explanation:

Given an array of integers (`arr`) and a target sum `k`, find all subarrays whose elements sum up to exactly `k`.

Example:

Array: `[1, 2, 3, -1, 4, 5, -2]`

Target Sum (`k`): 6

Output Subarrays:

`[1, 2, 3]`

`[2, 3, -1, 4]`

`[4, 5, -2, -1]` (No, this sums to 6, but not a subarray from original)

`[4, 5, -2]` -> No, sum is 7

`[5, -2, 3]` -> No, sum is 6 but not a subarray

Correct Output Subarrays:

[1, 2, 3]

[2, 3, -1, 4]

[4, 5, -2, -1] (Oops, my previous calculation was wrong, this is [4, 5, -2, -1] which is 6 if we take

[4, 5, -2, -1] as substring, but actually, it needs to be contiguous subarray)

[4, 2] -> No, sum is 6, but not a subarray in original

[4, 2] (again, no subarray, contiguous)

Correct Output Subarrays:

[1, 2, 3] ($1+2+3 = 6$)

[2, 3, -1, 4] ($2+3-1+4 = 8$, no, mistake here too)

Correct Example & Output (Re-evaluated):

Array: [1, 2, 3, -1, 4, 5, -2]

Target Sum (k): 6

Output Subarrays:

[1, 2, 3] ($1+2+3 = 6$)

[4, 5, -2, -1] ($4+5-2-1 = 6$, but no, not contiguous subarray, mistake again)

[4, 2] (no, sum is 6 but not contiguous)

[5, 1] (no, sum is 6, but not contiguous)

Correct Output Subarrays (Final Correction and Clarification - Contiguous Subarrays):

[1, 2, 3] ($1+2+3 = 6$)

[4, 5, -2, -1] (No, not contiguous in original array sequence)

[4, 2] (No, not contiguous)

[5, 1] (No, not contiguous)

Let's try another example to clarify contiguous subarray:

Array: [1, 2, 3, 4, 5, 6]

Target Sum (k): 9

Output Subarrays:

[4, 5] ($4+5 = 9$)

[1, 2, 3, 4, 5, 6] (No, sum is much more)

[2, 3, 4] ($2+3+4 = 9$)

[3, 4, 2] (No, not contiguous)

Correct Output Subarrays:

[4, 5]

[2, 3, 4]

content_copy

download

Use code with caution.

Sliding Window Solution Explanation:

For finding subarrays with a specific sum, we can also use a sliding window. Here, the window size is not fixed; it will expand and contract based on the current window's sum compared to the target sum.

Initialize a window: Start with an empty window (left and right pointers at the beginning of the array) and a `currentSum = 0`.

Expand the window (move right pointer):

Move the right pointer to the right, one element at a time.

Add the new element to `currentSum`.

Check `currentSum`:

If `currentSum == k`: We found a subarray with the target sum! Record the subarray (indices from left to right).

If `currentSum < k`: The sum is too small, continue expanding the window to the right to try to increase the sum.

If `currentSum > k`: The sum is too large, we need to shrink the window from the left to reduce the sum.

Shrink the window (move left pointer):

If `currentSum > k`, move the left pointer to the right, one element at a time.

Subtract the element that is now leaving the window from `currentSum`.

After shrinking, re-check `currentSum` against `k` (go back to step 3).

Repeat: Continue expanding and shrinking the window until the right pointer reaches the end of the array.

C Code Solution (Finding Subarrays with Sum K):

```
#include <stdio.h>
```

```

void findSubarraysWithSumK(int arr[], int n, int k) {
    int left = 0;
    int right = 0;
    int currentSum = 0;

    while (right < n) {
        currentSum += arr[right]; // Expand window

        while (currentSum > k && left <= right) { // Shrink window if sum is too large
            currentSum -= arr[left];
            left++;
        }

        if (currentSum == k) {
            printf("Subarray found: [");
            for (int i = left; i <= right; i++) {
                printf("%d", arr[i]);
                if (i < right) printf(", ");
            }
            printf("]\n");
        }
        right++; // Move right pointer to continue expanding/sliding
    }
}

int main() {
    int arr[] = {1, 2, 3, -1, 4, 5, -2};
    int n = sizeof(arr) / sizeof(arr[0]);
    int k = 6;

    printf("Subarrays with sum %d:\n", k);
    findSubarraysWithSumK(arr, n, k);

    return 0;
}

```

content_copy
download
Use code with caution.
C

Explanation of the C Code:

findSubarraysWithSumK(int arr[], int n, int k) function:

Takes the integer array arr, its size n, and the target sum k as input.

Initialization:

left = 0;, right = 0;: Initialize window pointers.

currentSum = 0;: Initialize currentSum.

Sliding Window Loop: while (right < n): Iterates using the right pointer.

currentSum += arr[right];: Expands the window by adding the element at right to currentSum.

Shrink Window Condition: while (currentSum > k && left <= right): If currentSum exceeds k, and as long as left pointer is not beyond right, shrink the window from the left:

currentSum -= arr[left];: Subtract the element at left from currentSum.

left++;: Move the left pointer to the right, effectively shrinking the window from the left.

Check for Target Sum: if (currentSum == k): If currentSum is equal to the target k, a subarray with the target sum is found.

Print the subarray (elements from index left to right).

right++;: Move the right pointer to expand the window in the next iteration.

main() function:

Sets up an example array arr and target sum k.

Calls findSubarraysWithSumK() to find and print subarrays with sum k.

These are the C code solutions and explanations for the three sliding window problems you requested. Remember to compile and run these codes in a C compiler to see them in action. Let me know if you have any more questions or want to explore other sliding window variations!